

TriChannel - Supplementary Code and Data

Anonymized code, per-trajectory score caches, benchmark data, and result files for the IEEE SaTML 2027 submission "TriChannel: Signal-Support-Aware Fusion for Prompt-Injection Detection in LLM Agents".

Every table and figure in the paper regenerates from this folder with ZERO API CALLS: the per-trajectory channel scores are shipped in results/cache/, the pattern channel is a local deterministic verifier, and each analysis script re-derives its statistics (bootstrap CIs, McNemar tests, fusion rules, learned heads) from those inputs.

Documents in this artifact

document	contents
README.pdf	this file: layout, table-to-file map, cache format, providers
PLAYBOOK.pdf	step-by-step runbook: order, commands, expected numbers, runtimes, verification gates, optional live re-scoring
RESULTS.pdf	every published number and all five figures, generated directly from the result JSONs by make_RESULTS.py

RESULTS.pdf is produced by make_RESULTS.py, which loads the JSONs in results/ at render time - no numbers are transcribed by hand anywhere in that document.

Repository layout

path

run_*.py
schemas.py
adaptive/
benchmarks/
baselines/

pipeline/
evaluation/
taxonomy/
osi_bench/
tests/
results/
results/cache/
figs/
data/InjecAgent/
data/bipia/
agentdojo_clone/runs/
make_README.py /
make_PLAYBOOK.py /
make_RESULTS.py

contents

experiment drivers (one per paper section - see map)
trajectory / tool-call / decision dataclasses
adaptive paraphrase generator (seed 0, 440 draws)
InjecAgent / AgentDojo / tool-call loaders
channels: glm_guard (text), camel_full (provenance),
deterministic (pattern, 48 signatures), comparators
conformal verifier, tool gateway, layered system
AUC, TPR@FPR, paired bootstrap, McNemar, Holm, Kuncheva
capability + sensitivity + threat-taxonomy YAML
base trajectories + mutations for the tool-call bench
unit tests (pytest)
result JSONs backing every paper table
per-trajectory score caches (see format below)
figure scripts + rendered PDFs
public InjecAgent release (Zhan et al. 2024)
public BIPIA corpus subset (Yi et al. 2023)
AgentDojo shipped runs, gpt-3.5-turbo-0125 subset
generators for the three PDF documents

Paper table / figure to artifact map

paper item	result file	script
Fig. 3a regime heatmap; VI-B table	run_or_extra_comparison.json +	run_or_extra_comparison.py,
	run_camel_full_eval.json,	run_camel_full_eval.py,
	run_toolcall_bench.json,	run_toolcall_bench.py
	run_agentdojo_matched_fpr.json	run_ensemble_ablation.py
Fig. 3b + Table III ablation; VI-C	run_ensemble_ablation.json	run_ensemble_ablation.py
VI-E learned fusion + rules	run_fusion_rule_ablation.json	run_fusion_rule_ablation.py
VI-F transfer + paired-diff CIs	run_ensemble_ablation.json +	run_ensemble_ablation.py,
	run_paired_head_diffs.json	run_paired_head_diffs.py
VI-D backbone deconfound (live only)	run_backbone_deconfound.json (+_injecagent)	run_backbone_deconfound.py
VI-C tighter operating points (1% FPR)	run_tpr_at_1pct.json	run_tpr_at_1pct.py
VI-G composition ceiling	run_review_response.json	run_review_response.py
VI-H contamination pre/post	run_contamination_prepost.json	run_contamination_prepost.py
VI-I adaptive fused	run_adaptive_fusion_eval_glm.json (+.log)	run_adaptive_fusion_eval.py
VI-I adaptive McNemar detail	run_adaptive_mcnemar.json	run_adaptive_mcnemar.py
VI-J cost/latency (Table VIII)	run_cost_table.json	run_cost_table.py
VI-I circularity split (Fig. 5b)	run_taxonomy_circularity.json	run_taxonomy_circularity.py
VI-I layer bypass rates	run_adaptive_layer_robustness.json	run_adaptive_layer_robustness.py
statistical treatment	run_ablation_ci.json,	run_ablation_ci.py,
	run_diversity_stats.json,	
	all_mcnemar_counts.json	run_diversity_stats.py
IV-C cheap-feature chance	run_signal_probe.json	run_signal_probe.py
VI-K enforcement layer	run_blast_radius_eval.json	run_blast_radius_eval.py
figures	figs/*.pdf	figs/make_figs.py,
		figs/make_figs2.py

Cache format, providers, third-party data

Cache format

Each `results/cache/<channel>__<benchmark>.jsonl` file holds one JSON object per trajectory with a positional `idx` key matching the benchmark iteration order, plus the channel output (score for text, mal for provenance). The pattern channel is always recomputed locally (deterministic, ~27 ms/trajectory, see `run_cost_table.json`) rather than cached. Because keying is positional, analysis scripts assert row-count alignment against the rebuilt benchmark before reading.

Providers (disclosure)

Static benches: text and CaMeL P/Q on Claude Haiku 4.5 via OpenRouter; comparators GLM-4.6, GLM-5.2, GPT-4o-mini, Claude Haiku 4.5 under one guard prompt, Llama Guard 4 shipped prompt.

Adaptive bench and contamination (paper VI-H/VI-I): text and CaMeL on GLM-4.6 served by z.ai. The z.ai endpoint silently aliases `claude-*` model ids to GLM models; every run was probe-gated before spend and the configuration is labeled GLM-4.6 throughout.

Backbone deconfound (paper VI-B note): only the provenance channel was re-scored on Gemma 4 31B via an OpenAI-compatible endpoint, on both the tool-call and the InjecAgent bench; per-trajectory scores ship in `results/cache/ (camel_gw_*_*.jsonl)`, so the analysis reruns with no credentials.

No credentials are required anywhere in this folder; re-scoring against live endpoints is optional (see `PLAYBOOK.pdf`).

Third-party data

`data/InjecAgent/` and `agentdojo_clone/runs/` are redistributed subsets of the public InjecAgent and AgentDojo releases, included so benchmark rebuilds (and the pre-fix contamination construction) reproduce without external downloads. They belong to their original authors (Zhan et al. 2024; Debenedetti et al. 2024).

Determinism

Benchmarks are seeded (`seed=0` everywhere). `run_adaptive_fusion_eval.py` asserts that a second benchmark rebuild reproduces every trajectory id before any scoring; the same assertion pattern guards the circularity split.